Docker Containerization Project

Short Technical Report

March 2026

Name – Mayank Thakur
Sap id -500123412
Roll no –R2142230924

1. Introduction

This report documents the design and implementation of a multi-container Docker application, covering architecture decisions, build optimizations, network configuration, and storage persistence. The project demonstrates best practices for containerizing a backend application alongside a database service using Docker and Docker Compose.

2. Build Optimization Explanation

Optimizing Docker image builds is critical for reducing build time, minimizing image size, and improving security. The following strategies were applied:

2.1 Multi-Stage Builds

Multi-stage builds allow separation of the build environment from the runtime environment. In the backend Dockerfile, the first stage installs all dependencies and compiles/builds the application. The second (final) stage copies only the built artifacts, resulting in a significantly smaller production image.

Example structure:

```
FROM node:18-alpine AS builder
WORKDIR /app
```

```
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/index.js"]
```

2.2 Layer Caching

Docker caches each instruction layer. By copying package.json before the full source code, dependency installation is only re-run when dependencies change, not on every code edit. This dramatically reduces rebuild times during development.

2.3 Using Alpine Base Images

Alpine Linux base images (e.g., node:18-alpine, python:3.11-alpine) are minimal (~5MB), compared to standard Debian-based images (~300MB+). This reduces attack surface and image download time.

2.4 .dockerignore File

A .dockerignore file excludes unnecessary files (node_modules, .git, logs, test files) from the build context, speeding up the Docker daemon's processing.

3. Image Size Comparison

The table below shows the impact of build optimizations on image size:

Image	Base Image	Unoptimized Size	Optimized Size
Backend	node:18	~900 MB	~160 MB
Database	postgres:15	~400 MB	~250 MB
Total	—	~1.3 GB	~410 MB

Optimized images are approximately 68% smaller, reducing storage costs and deployment time.

4. Network Design Diagram

The application uses a custom Docker bridge network to enable container-to-container communication while isolating services from external access.

4.1 Network Architecture

The network topology is described below (a visual diagram is included in the accompanying screenshot proofs):

Component	Details
Network Name	app_network
Network Driver	Bridge (custom)
Backend Container IP	172.20.0.2
Database Container IP	172.20.0.3
Exposed Port (Host)	8080 -> 3000 (backend)
Internal DB Port	5432 (not exposed externally)

4.2 Network Creation Command

The custom network was created using the following command:

```
docker network create --driver bridge --subnet 172.20.0.0/16 app_network
```

Containers were connected to this network via docker-compose.yml, ensuring the backend can reach the database using its service name (db) as the hostname, without hardcoding IP addresses.

5. macvlan vs ipvlan Comparison

Both macvlan and ipvlan are advanced Docker network drivers that allow containers to appear as physical devices on the host network. They are typically used in production environments requiring direct Layer 2/3 access.

Feature	macvlan	ipvlan
MAC Address	Each container gets a unique MAC address	Containers share the host MAC address
Layer	Operates at Layer 2 (Data Link)	Operates at Layer 2 (L2) or Layer 3 (L3)
ARP Overhead	Higher — each container sends ARP requests	Lower — shared MAC reduces ARP traffic
Switch Requirement	Switch must allow multiple MACs per port	No special switch config needed
Container-to-Host	Not directly reachable (promiscuous issue)	Can communicate with host (L3 mode)
Use Case	VMs-like isolation, legacy apps needing unique MACs	High-performance, cloud environments
Performance	Slightly lower (more ARP)	Higher (less overhead)
Complexity	Moderate	Low to moderate

5.1 Recommendation

For this project, a custom bridge network was used, as macvlan and ipvlan are best suited for production bare-metal or VM deployments. If direct host-network access were required, ipvlan (L3 mode) would be preferred over macvlan due to lower overhead and simpler switch configuration.

6. Volume Persistence

Data persistence is achieved using Docker named volumes, which survive container restarts and re-creation. The database container mounts a volume to its data directory, ensuring that database files are not lost when the container is stopped or rebuilt.

Volume definition in docker-compose.yml:

```
volumes:
  db_data:

services:
  db:
```

```
    image: postgres:15-alpine
    volumes:
      - db_data:/var/lib/postgresql/data
```

Persistence was verified by inserting test data into the database, stopping and removing the container (docker compose down), then restarting it (docker compose up) and confirming the data remained intact.

7. Conclusion

This project successfully demonstrated containerization best practices including multi-stage Dockerfile builds, custom bridge networking, and volume persistence. Key outcomes include:

- Image size reduced by ~68% through multi-stage builds and Alpine base images
- Secure inter-container communication via a custom Docker bridge network
- Database data persisted reliably across container restarts using named volumes
- Clear understanding of macvlan vs ipvlan network driver trade-offs

All deliverables — Dockerfiles, docker-compose.yml, network commands, and screenshot proofs — are available in the project GitHub repository.